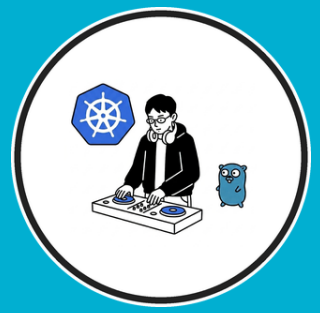


# アルゴリズム特論

## 第5章　高階関数

2581008 井上 裕介

# 今日の流れ（110分）

| 時間 | 内容 |
|---|---|
| 0:00–0:05 | 導入・第4章の振り返り |
| 0:05–0:20 | 5.1 カリー化関数 |
| 0:20–0:50 | 5.2 高階実演 / 5.3 道具箱 / 5.4 ラムダ式 |
| 0:50–1:00 | 前半の演習（課題 問1） |
| 1:05–1:30 | 5.5 畳み込み / 5.6 `$` |
| 1:30–1:42 | 5.7 関数合成 |
| 1:42–1:50 | 課題の説明 |

# 目次


5.1 カリー化関数

5.2 高階実演

5.3 関数プログラマの道具箱

5.4 ラムダ式

5.5 畳み込み、見込みアリ！

5.6 `$` を使った関数適用

5.7 関数合成

# 第4章の再帰を振り返る

```haskell
maximum' :: (Ord a) => [a] -> a
maximum' [] = error "empty list"
maximum' [x] = x
maximum' (x:xs) = max x (maximum' xs)
```

```haskell
reverse' :: [a] -> [a]
reverse' [] = []
reverse' (x:xs) = reverse' xs ++ [x]
```

再帰関数は、再帰せずに答えが決まる基底部（`maximum' [x] = x` など）と、問題を小さく分けて自分自身を呼ぶ再帰部からできている。リストは `[]` と `x:xs` に分けて考える

今日は、リストを先頭から1つずつ処理するこの形を、高階関数を使って短く書く

HANDS-ON

# H0　GHCi で課題ファイルを読み込む

0:03–0:05

1. devcontainer のターミナルで作業ディレクトリへ移動し、GHCi で課題ファイルを開く

```
$ cd lectures/section05
$ ghci report5.hs
```

2. GHCi でよく使うコマンドは次の4つ

| コマンド | 意味 |
|---|---|
| `:l ファイル名` | ファイルを読み込む（load） |
| `:r` | 編集したファイルを再読み込み（reload） |
| `:t 式` | 式の型を表示（type） |
| `:q` | GHCi を終了 |

3. `runghc report5.hs` で課題のテストを一括で実行する（未実装の問は `[--]` と表示される）

# 目次


5.1 カリー化関数

5.2 高階実演

5.3 関数プログラマの道具箱

5.4 ラムダ式

5.5 畳み込み、見込みアリ！

5.6 `$` を使った関数適用

5.7 関数合成

# 5.1　カリー化関数

- **Haskell のすべての関数は引数を1つしかとらない**

```
max 4 5
(+) 4 5
(++) [1] [2]
```

上から順に、2つを比べて大きい方を返し、2つを足し、2つのリストを結合する。どれも引数を2つとるように見えるが、実際はカリー化された関数である

Haskell は、複数の引数をとる関数を、引数を1つ受け取って関数を返す関数の連なりとして扱う。これをカリー化と呼ぶ。また、関数を引数に取ったり返り値として返したりする関数を高階関数と呼ぶ

# 5.1　カリー化関数

- `max 4 5` は `(max 4) 5` と同じ

```
ghci> max 4 5
5
ghci> (max 4) 5
5
ghci> :t max
max :: Ord a => a -> a -> a
```

1. `max 4` は、4 と比べて大きい方を返す関数を返す
2. その関数を `5` に適用する

型の `a -> a -> a` は `a -> (a -> a)` と読む

関数を本来より少ない引数で呼び出すと、残りの引数をとる関数が得られる。これを **部分適用** と呼ぶ

# 5.1　カリー化関数

- `multThree` を部分適用する

```
multThree :: Int -> Int -> Int -> Int
multThree x y z = x * y * z
```

型は `Int -> (Int -> (Int -> Int))` と読む。スペース区切りの関数適用は左結合なので、`multThree 3 5 9` は `((multThree 3) 5) 9` と同じになる

| 式 | 得られるもの |
| --- | --- |
| `multThree 3` | 3 と残り2つの引数を掛ける関数 |
| `(multThree 3) 5` | 3 と 5 と残り1つの引数を掛ける関数 |
| `((multThree 3) 5) 9` | `135` |



関数を、材料を受け取って品物を作る工場に例えると、3 だけを入れたときに出てくるのは数ではなく、残りの材料を待つ小さな工場になる

# 5.1　カリー化関数

- **部分適用で新しい関数を作る**

```
ghci> let multTwoWithNine = multThree 9
ghci> multTwoWithNine 2 3
54
```

```
compareWithHundred :: Int -> Ordering
compareWithHundred x = compare 100 x
```

両辺の右端の `x` は省略できる

```
compareWithHundred = compare 100
```

`multThree 9` は 2 引数の関数で、残り2つを掛けて 9 倍する

`compare 100` は、数を受け取って 100 と比較する関数になる。`compare` の型 `(Ord a) => a -> (a -> Ordering)` に 100 を与えただけなので、型宣言は変えずに済む

```
ghci> compareWithHundred 99
GT
```

# 5.1　カリー化関数

- **中置関数はセクションで部分適用する**

```haskell
divideByTen :: (Floating a) => a -> a
divideByTen = (/10)

isUpperAlphanum :: Char -> Bool
isUpperAlphanum = (`elem` ['A'..'Z'])
```

```
ghci> divideByTen 200
20.0
```

中置関数の片側だけに値を置いて括弧で囲むと、値を置かなかった側を引数にとる関数になる。これをセクションと呼ぶ

`(/10) 200` ＝ `200 / 10`

`(10/) 200` ＝ `10 / 200`

注意：`(-4)` はマイナス4という数を表す。4 を引く関数は `(subtract 4)` と書く

# 5.1　カリー化関数

- **関数そのものは表示できない**

```
ghci> multThree 3 4
<interactive>:1:1: error:
    • No instance for (Show (Int -> Int)) arising from a use of ‘print’
        (maybe you haven't applied a function to enough arguments?)
```

`multThree 3 4` は `Int -> Int` 型の関数を返す。関数は `Show` 型クラスのインスタンスではないので、GHCi は結果を文字列にできない。`1 + 1` なら、結果の `2` に `show` を呼んで `"2"` を表示できる

後半で扱う畳み込みと関数合成は、どちらも部分適用を前提にしている

# 5.1　Go で書くカリー化と部分適用

カリー化は、`f(x, y)` を `f(x)(y)` の形に変換すること

```
func add(x, y int) int {
    return x + y
}

func curriedAdd(x int) func(int) int {
    return func(y int) int {
        return x + y
    }
}

fmt.Println(add(2, 3))
fmt.Println(curriedAdd(2)(3))
```

どちらも 5 を表示する

部分適用は、引数の一部を固定して、残りを受け取る関数を作ること

```
add2 := curriedAdd(2)
fmt.Println(add2(3))
fmt.Println(add2(10))
```

カリー化された関数なら、引数を1つ渡すだけで部分適用になる（5 と 12 を表示）。カリー化していない `add` では、無名関数で包む必要がある

```
add2 := func(y int) int { return add(2, y) }
```

Haskell の関数は最初からカリー化されているので、`(+) 2` と書くだけで部分適用になる

HANDS-ON

# H1　部分適用とセクションを試す

3分

`ghci handson.hs` で起動し、次を入力する。結果を予想してから確かめる

```
ghci> :t max
ghci> :t max 4
ghci> (max 4) 5
ghci> multTwoWithNine 2 3
ghci> (/10) 200
ghci> (10/) 200
ghci> (-4)
ghci> (subtract 4) 10
ghci> multThree 3 4
```

`:t max 4` で型がどう変わるか、`(10/)` で左右を入れ替えると何が起きるか、`multThree 3 4` がエラーになる理由を考える

# 目次


5.1 カリー化関数

**5.2 高階実演**

5.3 関数プログラマの道具箱

5.4 ラムダ式

5.5 畳み込み、見込みアリ！

5.6 `$` を使った関数適用

5.7 関数合成

# 5.2　高階実演

- **関数を受け取り、2回適用する `applyTwice`**

```
applyTwice :: (a -> a) -> a -> a
applyTwice f x = f (f x)
```

```
ghci> applyTwice (+3) 10
16
ghci> applyTwice (++ " HAHA") "HEY"
"HEY HAHA HAHA"
ghci> applyTwice ("HAHA " ++) "HEY"
"HAHA HAHA HEY"
ghci> applyTwice (multThree 2 2) 9
144
ghci> applyTwice (3:) [1]
[3,3,1]
```

型の `(a -> a)` の括弧は省けない。この括弧で、第1引数が `a` を受け取って `a` を返す関数であることを表している。`->` は右結合なので、括弧がないと `a -> a -> a -> a`（3引数関数）になってしまう

1引数の関数を渡したいときは、`(+3)` や `multThree 2 2` のように部分適用で作って渡す

# 5.2　高階実演

- `zipWith` を実装する

```
zipWith' :: (a -> b -> c) -> [a] -> [b] -> [c]
zipWith' _ [] _ = []
zipWith' _ _ [] = []
zipWith' f (x:xs) (y:ys)
    = f x y : zipWith' f xs ys
```

第1引数は2引数の関数 `a -> b -> c`、残りの2つは `[a]` と `[b]` で、結果は `[c]` になる

どちらかのリストが空になったら終わる。

形は `zip` と同じで、`(x, y)` の代わりに `f x y` を返す

型がよく分からなくなったら、型宣言を省いて GHCi の `:t` で推論させるとよい

# 5.2　高階実演

- `zipWith'` の使用例

```
ghci> zipWith' (+) [4,2,5,6] [2,6,2,3]
[6,8,7,9]
ghci> zipWith' max [6,3,2,1] [7,3,1,5]
[7,3,2,5]
ghci> zipWith' (++) ["foo ", "bar ", "baz "] ["fighters", "hoppers", "aldrin"]
["foo fighters","bar hoppers","baz aldrin"]
ghci> zipWith' (*) (replicate 5 2) [1..]
[2,4,6,8,10]
ghci> zipWith' (zipWith' (*)) [[1,2,3],[3,5,6],[2,3,4]] [[3,2,2],[3,4,5],[5,4,3]]
[[3,4,6],[9,20,30],[10,12,12]]
```

渡す関数を替えるだけで、同じ `zipWith'` が足し算、最大値、文字列の連結、2次元リストの要素ごとの積を計算する

# 5.2　高階実演

- 最初の2つの引数を入れ替える `flip` を実装する

```
flip' :: (a -> b -> c) -> (b -> a -> c)
flip' f = g
    where g x y = f y x
```

```
flip' :: (a -> b -> c) -> b -> a -> c
flip' f y x = f x y
```

右はカリー化を使って短く書いたもので、2つは同じ関数になる

```
ghci> zip [1,2,3,4,5] "hello"
[(1,'h'),(2,'e'),(3,'l'),(4,'l'),(5,'o')]
ghci> flip' zip [1,2,3,4,5] "hello"
[('h',1),('e',2),('l',3),('l',4),('o',5)]
ghci> zipWith (flip' div) [2,2..] [10,8,6,4,2]
[5,4,3,2,1]
```

最後の例では `flip' div 2 10` が `div 10 2` として計算される

# 目次


5.1 カリー化関数

5.2 高階実演

**5.3 関数プログラマの道具箱**

5.4 ラムダ式

5.5 畳み込み、見込みアリ！

5.6 `$` を使った関数適用

5.7 関数合成

# 5.3　関数プログラマの道具箱

- `map` はリストのすべての要素に関数を適用する

```
map :: (a -> b) -> [a] -> [b]
map _ [] = []
map f (x:xs) = f x : map f xs
```

```
ghci> map (+3) [1,5,3,1,6]
[4,8,6,4,9]
ghci> map (++ "!") ["BIFF", "BANG", "POW"]
["BIFF!","BANG!","POW!"]
ghci> map (replicate 3) [3..6]
[[3,3,3],[4,4,4],[5,5,5],[6,6,6]]
ghci> map (map (^2)) [[1,2],[3,4,5,6],[7,8]]
[[1,4],[9,16,25,36],[49,64]]
ghci> map fst [(1,2),(3,5),(6,3),(2,6),(2,5)]
[1,3,6,2,2]
```

`a` から `b` への関数と `a` のリストを受け取り、`b` のリストを返す

`map (+3) [1,5,3,1,6]` は `[x+3 | x <- [1,5,3,1,6]]` と同じ結果になる。`map (map (^2))` のように入れ子にする場面では、リスト内包表記より `map` のほうが読みやすい

# 5.3　関数プログラマの道具箱

- `filter` は述語を満たす要素だけを残す

```
filter :: (a -> Bool) -> [a] -> [a]
filter _ [] = []
filter p (x:xs)
    | p x       = x : filter p xs
    | otherwise = filter p xs
```

`True` か `False` を返す関数を述語と呼ぶ

`p x` が `True` なら新しいリストに含め、`False` なら含めない

```
ghci> filter (>3) [1,5,3,2,1,6,4,3,2,1]
[5,6,4]
ghci> filter even [1..10]
[2,4,6,8,10]
ghci> filter (`elem` ['a'..'z']) "u LaUgH aT mE BeCaUsE I aM diFfeRent"
"uagameasadifeent"
```

# 5.3　関数プログラマの道具箱

- `map`・`filter` とリスト内包表記

```
ghci> filter (<15) (filter even [1..20])
[2,4,6,8,10,12,14]
ghci> [x | x <- [1..20], x < 15, even x]
[2,4,6,8,10,12,14]
```

どちらを使うかに決まりはないので、読みやすいほうを選ぶ

第4章のクイックソートも `filter` で書ける

```
quicksort :: (Ord a) => [a] -> [a]
quicksort [] = []
quicksort (x:xs) =
    let smallerOrEqual = filter (<= x) xs
        larger = filter (> x) xs
    in  quicksort smallerOrEqual
          ++ [x] ++ quicksort larger
```

# 5.3　関数プログラマの道具箱

- **10万以下で 3829 で割り切れる最大の数を求める**

```haskell
largestDivisible :: Integer
largestDivisible = head (filter p [100000,99999..])
    where p x = x `mod` 3829 == 0
```

```
ghci> largestDivisible
99554
```

降順のリストを `filter` するので、条件を満たす最大の数が先頭に現れる。Haskell は遅延評価なので、`head` が最初の解を受け取った時点で評価が止まる。そのため、元のリストが有限でも無限でも同じように書ける

# 5.3　関数プログラマの道具箱

- `takeWhile` は条件を満たす間だけ要素を取り出す

```
ghci> takeWhile (/=' ') "elephants know how to party"
"elephants"
```

10000 より小さい奇数の平方数の和を求める

```
ghci> sum (takeWhile (<10000) (filter odd (map (^2) [1..])))
166650
ghci> sum (takeWhile (<10000) [m | m <- [n^2 | n <- [1..]], odd m])
166650
```

1つ目の式は、無限リスト `[1..]` に `map (^2)`、`filter odd`、`takeWhile (<10000)`、`sum` の順に関数を適用している。`takeWhile` が途中で切るので、無限リストから始めても計算は終わる

# 5.3　関数プログラマの道具箱

## コラッツ列

1. 任意の自然数から始める
2. 数が 1 になったら終わる
3. 偶数なら 2 で割る
4. 奇数なら 3 倍して 1 を足す
5. 新しい値で繰り返す

```
chain :: Integer -> [Integer]
chain 1 = [1]
chain n
    | even n = n : chain (n `div` 2)
    | odd n  = n : chain (n * 3 + 1)
```

```
ghci> chain 10
[10,5,16,8,4,2,1]
ghci> chain 1
[1]
ghci> chain 30
[30,15,46,23,70,35,106,53,
 160,80,40,20,10,5,16,8,4,2,1]
```

どの列も 1 で終わるので、基底部は `1` にする

では、1〜100 の数のうち、長さが 15 より長いコラッツ列の開始数はいくつあるか

# 5.3　関数プログラマの道具箱

## ● 長いコラッツ列を数える `numLongChains`

```haskell
numLongChains :: Int
numLongChains = length (filter isLong (map chain [1..100]))
    where isLong xs = length xs > 15
```

1. `[1..100]` を `chain` で `map` し、コラッツ列のリストを得る
2. 長さが 15 より長いかを調べる述語 `isLong` で `filter` する
3. 残った列の個数を `length` で数えると `66` になる

※ 教科書の問題文は「長さ15以上」だがコードは `> 15`。本講義はコードに合わせ、15より長いとする

※ `length` は `Int` を返す。一般の `Num a` にしたいなら `fromIntegral` を使う

# 5.3　関数プログラマの道具箱

- `map` に複数の引数をとる関数を与える

```
ghci> let listOfFuns = map (*) [0..]
ghci> (listOfFuns !! 4) 5
20
```

`(*)` は2引数関数なので、1つだけ与えると1引数関数になる。そのため `map (*) [0..]` は `[(0*), (1*), (2*), (3*), (4*), ...]` という関数のリストになる

`listOfFuns !! 4` で取り出した `(4*)` を 5 に適用するので、`4 * 5` で 20 になる

# 目次


5.1 カリー化関数

5.2 高階実演

5.3 関数プログラマの道具箱

**5.4 ラムダ式**

5.5 畳み込み、見込みアリ！

5.6 `$` を使った関数適用

5.7 関数合成

# 5.4　ラムダ式

- 1回だけ必要な関数を作る無名関数

```
numLongChains :: Int
numLongChains = length (filter (\xs -> length xs > 15)
                               (map chain [1..100]))
```

ラムダ式は `\`、引数、`->`、本体の順に書き、普通は全体を括弧で囲む。`\` はギリシャ文字のラムダλに見立てた記号である

ここでは、`where` で `isLong` を定義する代わりに、その場で関数を作って渡している

ただし、ラムダ式が要らない場面もある。

次の2つは同じ結果になり、部分適用で書いた1行目のほうが読みやすい

```
map (+3) [1,6,3,2]
map (\x -> x + 3) [1,6,3,2]
```

# 5.4　ラムダ式

- **複数の引数をとるラムダ式とパターンマッチ**

```
ghci> zipWith (\a b -> (a * 30 + 3) / b) [5,4,3,2,1] [1,2,3,4,5]
[153.0,61.5,31.0,15.75,6.6]
ghci> map (\(a,b) -> a + b) [(1,2),(3,5),(6,3),(2,6),(2,5)]
[3,8,9,8,7]
```

```
addThree' :: Int -> Int -> Int -> Int
addThree' = \x -> \y -> \z -> x + y + z

flip' :: (a -> b -> c) -> b -> a -> c
flip' f = \x y -> f y x
```

ラムダ式にはパターンを1つしか書けない。マッチに失敗するとランタイムエラーになる

`flip'` をラムダ式で書くと、新しい関数を作って返していることが読み取りやすくなる

# 5.4　Go と Julia で書くラムダ式と map・filter

1〜10 の偶数を2乗したリストを作る。Haskell では `map (\x -> x * x) (filter even [1..10])` と書く

Go（標準に map・filter がないので定義する）

```go
func Filter[T any](xs []T, p func(T) bool) (ys []T) {
    for _, x := range xs {
        if p(x) {
            ys = append(ys, x)
        }
    }
    return
}

func Map[T, U any](xs []T, f func(T) U) (ys []U) {
    for _, x := range xs {
        ys = append(ys, f(x))
    }
    return
}
```

```go
xs := []int{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
evens := Filter(xs, func(x int) bool { return x%2 == 0 })
squares := Map(evens, func(x int) int { return x * x })
```

Julia

```julia
squares = map(x -> x * x, filter(iseven, 1:10))
```

どちらも `[4, 16, 36, 64, 100]` になる。Go の `func(x int) int { return x * x }` と Julia の `x -> x * x` が、Haskell の `\x -> x * x` にあたる

EXERCISE

# 前半の演習（課題 問1）

10分

課題は `report5.hs` の `undefined` を書き換え、`:r` と `runghc report5.hs` で確かめる

| | 内容 | ヒント |
|---|---|---|
| 課題 問1 | `zipWith'` を実装 | どちらかのリストが `[]` なら終わる |
| 練習 | `flip'` を実装 | `flip' f y x = ...` |
| 練習 | `chain` と `numLongChains` | `map`、`filter`、`length` の順に使う |

```
$ runghc report5.hs
[OK] 問1 zipWith'
[--] 問2 map'  (未実装またはエラー)
...
```

練習は提出しない。`ghci` で試し、休憩明けの解答例で答え合わせをする

# 休憩 5分

再開は 1:05 から

後半は畳み込み、`$`、関数合成を扱う

# 前半の演習の解答例

```haskell
zipWith' :: (a -> b -> c) -> [a] -> [b] -> [c]
zipWith' _ [] _ = []
zipWith' _ _ [] = []
zipWith' f (x:xs) (y:ys) = f x y : zipWith' f xs ys

flip' :: (a -> b -> c) -> b -> a -> c
flip' f y x = f x y

chain :: Integer -> [Integer]
chain 1 = [1]
chain n
    | even n = n : chain (n `div` 2)
    | odd n  = n : chain (n * 3 + 1)

numLongChains :: Int
numLongChains = length (filter (\xs -> length xs > 15) (map chain [1..100]))
```

# 目次


5.1 カリー化関数

5.2 高階実演

5.3 関数プログラマの道具箱

5.4 ラムダ式

**5.5 畳み込み、見込みアリ！**

5.6 `$` を使った関数適用

5.7 関数合成

# 5.5　畳み込み、見込みアリ！

## ● 畳み込み（fold）はリストを1つの値にまとめる

第4章の再帰関数の多くは、基底部を空リストにし、`x:xs` で先頭要素と残りに分けるという同じ形をしている

畳み込み関数は次の3つを受け取る

1. 2引数関数（`+` や `div` など）
2. アキュムレータ（途中の計算結果を持つ値）の初期値
3. 畳み込むリスト

リストを1要素ずつ1回だけ走査して何かを返す関数なら、畳み込みで書ける



# 5.5　畳み込み、見込みアリ！

- `foldl` は左から畳み込む

```
foldl :: (a -> b -> a) -> a -> [b] -> a
```

```
sum' :: (Num a) => [a] -> a
sum' xs = foldl (\acc x -> acc + x) 0 xs
```

```
ghci> sum' [3,5,2,1]
11
```

| acc | x | 残り | 新しい acc |
|---|---|---|---|
| 0 | 3 | [5,2,1] | 3 |
| 3 | 5 | [2,1] | 8 |
| 8 | 2 | [1] | 10 |
| 10 | 1 | [] | 11 |

まず初期値と先頭の要素に2引数関数を適用して新しいアキュムレータを作り、それと次の要素に同じ関数を適用する。これをリストの最後まで繰り返す

※ 今の GHCi で `:t foldl` とすると `Foldable t => (b -> a -> b) -> b -> t a -> b` と表示される。`t a` を `[a]` と読み替えれば同じ意味

# 5.5　畳み込み、見込みアリ！

- カリー化を使って `sum'` を短く書く

```
sum' :: (Num a) => [a] -> a
sum' xs = foldl (\acc x -> acc + x) 0 xs
```

ラムダ式 `\acc x -> acc + x` は `(+)` と同じなので、次のように書ける

```
sum' :: (Num a) => [a] -> a
sum' = foldl (+) 0
```

`foldl (+) 0` はリストを受け取る関数を返すので、両辺の `xs` を省略できる。一般に `foo a = bar b a` は、カリー化のおかげで `foo = bar b` と書ける

# 5.5　畳み込み、見込みアリ！

- **`foldr` は右から畳み込む**

```
foldl :: (a -> b -> a) -> a -> [b] -> a
foldr :: (a -> b -> b) -> b -> [a] -> b
```

```
map' :: (a -> b) -> [a] -> [b]
map' f xs = foldr (\x acc -> f x : acc) [] xs
```

```
map' f xs = foldl (\acc x -> acc ++ [f x]) [] xs
```

2引数関数の引数の順番が逆になる。`foldl` では `\acc x -> ...`、`foldr` では `\x acc -> ...` と書く

`map' (+3) [1,2,3]` は、右から `6:[]`、`5:[6]`、`4:[5,6]` の順に組み立てられる

`foldl` 版は `++` が毎回左側のリストをたどり直すので、長さ n のリストに $n^2$ に比例する時間がかかる。リストを作るときは普通 `foldr` と `:` を使う

※ foldr もリストは左から走査する。結果が右から畳んだように見える

# 5.5　畳み込み、見込みアリ！

- **`foldr` で `elem` を実装する**

```
elem' :: (Eq a) => a -> [a] -> Bool
elem' y ys = foldr (\x acc -> if x == y then True else acc) False ys
```

アキュムレータは真理値で、型は結果の型と同じになる。初期値を `False` にしておけば、空リストに対しても正しく `False` を返す。今の要素が探している値なら `True` を返し、そうでなければアキュムレータをそのまま返す

この例のように、アキュムレータには数だけでなく真理値やリストなど、どんな型の値も使える

# 5.5　畳み込み、見込みアリ！

- **初期値を省ける `foldl1` と `foldr1`**

```
maximum' :: (Ord a) => [a] -> a
maximum' = foldl1 max
```

`foldl1` と `foldr1` は、リストの先頭（または末尾）の要素を初期アキュムレータとして使うので、初期値を渡さなくてよい

`maximum'` は、要素とアキュムレータを `max` で比べ、大きいほうを新しいアキュムレータにする。ただし空リストに対して呼ぶとランタイムエラーになる（`foldl`・`foldr` なら空リストでも動く）

空リストに意味がない関数のときに `foldl1` / `foldr1` を使う

# 5.5　畳み込み、見込みアリ！

- **畳み込みで書ける関数の例**

```haskell
reverse' :: [a] -> [a]
reverse' = foldl (\acc x -> x : acc) []

product' :: (Num a) => [a] -> a
product' = foldl (*) 1
```

```haskell
filter' :: (a -> Bool) -> [a] -> [a]
filter' p = foldr
  (\x acc -> if p x then x : acc else acc) []

last' :: [a] -> a
last' = foldl1 (\_ x -> x)
```

`reverse'` は、左から順に要素をアキュムレータの先頭にくっつける。このラムダ式は引数の順番が逆なだけの `(:)` なので、`foldl (flip (:)) []` とも書ける

残りの3つも同じ考え方で読める。`product'` は 1 から掛け合わせ、`filter'` は述語を満たすときだけ先頭に付け、`last'` は常に今の要素で置き換える

# 5.5　畳み込み、見込みアリ！

## ● 畳み込みを式の形で見る

2引数関数 `f`、初期値 `z` で `[3,4,5,6]` を右畳み込みすると、次の式になる

```
f 3 (f 4 (f 5 (f 6 z)))
```

`f = (+)`、`z = 0` なら `3 + (4 + (5 + (6 + 0)))` になる

2引数関数 `g`、初期値 `z` で左畳み込みすると、次の式になる

```
g (g (g (g z 3) 4) 5) 6
```

`g = flip (:)`、`z = []` なら次の式になり、結果は `[6,5,4,3]` になる

```
flip (:) (flip (:) (flip (:) (flip (:) [] 3) 4) 5) 6
```

# 5.5　畳み込み、見込みアリ！

## ● 無限リストを畳み込む

```haskell
and' :: [Bool] -> Bool
and' xs = foldr (&&) True xs
```

```haskell
True && (False && (True && True))
```

```haskell
(&&) :: Bool -> Bool -> Bool
True  && x = x
False && _ = False
```

```
ghci> and' (repeat False)
False
```

`and' (repeat False)` を書き下すと `False && (False && (False && ...` になる

`&&` は最初の引数が `False` なら2つ目の引数を評価しない（遅延評価）。そのため、無限に続く残りを見ずに `False` を返す

> `foldr` は、2つ目の引数を評価しないことがある関数となら無限リストでも止まる。`foldl` はリストの最後まで進もうとするので止まらない

# 5.5　畳み込み、見込みアリ！

- **スキャンはアキュムレータの途中経過をすべて返す**

```
ghci> scanl (+) 0 [3,5,2,1]
[0,3,8,10,11]
ghci> scanr (+) 0 [3,5,2,1]
[11,8,3,1,0]
ghci> scanl1 (\acc x -> if x > acc then x else acc) [3,4,5,3,7,9,2,1]
[3,4,5,5,7,9,9,9]
ghci> scanl (flip (:)) [] [3,2,1]
[[],[3],[2,3],[1,2,3]]
```

`scanl` では最後の要素に、`scanr` では先頭の要素に最終結果が入る。`scanl1` と `scanr1` は、`foldl1` と `foldr1` のスキャン版である

# 5.5　畳み込み、見込みアリ！

- **平方根の和が 1000 を超えるのは何個目かを求める**

```haskell
sqrtSums :: Int
sqrtSums = length (takeWhile (<1000) (scanl1 (+) (map sqrt [1..]))) + 1
```

```
ghci> sqrtSums
131
ghci> sum (map sqrt [1..131])
1005.0942035344083
ghci> sum (map sqrt [1..130])
993.6486803921487
```

和の途中経過が必要なので、畳み込みではなくスキャンを使う。和は増え続けるので、走査をやめない `filter` ではなく `takeWhile` で切る。1000 より小さい和が x 個あれば、x + 1 個目で 1000 を超える

# 5.5　Go と Julia で書く畳み込み

`[3, 5, 2, 1]` の和を左畳み込みで求める。Haskell では `foldl (\acc x -> acc + x) 0 [3,5,2,1]` と書く

Go（標準に fold がないので定義する）

```go
func Fold[T, A any](xs []T, init A, f func(A, T) A) A {
    acc := init
    for _, x := range xs {
        acc = f(acc, x)
    }
    return acc
}

sum := Fold([]int{3, 5, 2, 1}, 0,
    func(acc, x int) int { return acc + x })
```

Julia

```julia
total = foldl((acc, x) -> acc + x, [3, 5, 2, 1]; init = 0)
```

どちらも 11 になる。初期値、アキュムレータを先に受け取る2引数関数、リストという3つの部品は `foldl` と同じ

Julia には `foldr` もあり、途中経過を返す `accumulate` は `scanl1` にあたる

HANDS-ON

# H4　畳み込みで書き直す（課題 問2）

5分

`report5.hs` の `map'` と `filter'` を `foldr` で実装し、`runghc report5.hs` で `[OK]` になることを確かめる。`maximum'` と `reverse'` は練習で、第4回は再帰で実装した

```
map'     :: (a -> b) -> [a] -> [b]
filter'  :: (a -> Bool) -> [a] -> [a]
maximum' :: (Ord a) => [a] -> a
reverse' :: [a] -> [a]
```

次の順番で考える

1. アキュムレータの型と初期値は何か（空リストのときの答え）
2. 要素1つとアキュムレータから、新しいアキュムレータをどう作るか
3. 左右どちらから畳むか（リストを作るなら `foldr` と `:`）

# 目次


5.1 カリー化関数

5.2 高階実演

5.3 関数プログラマの道具箱

5.4 ラムダ式

5.5 畳み込み、見込みアリ！

**5.6 `$` を使った関数適用**

5.7 関数合成

# 5.6 `$` を使った関数適用

## 関数適用演算子 `$`

```
($) :: (a -> b) -> a -> b
f $ x = f x
```

```
ghci> sum (filter (> 10) (map (*2) [2..10]))
80
ghci> sum $ filter (> 10) (map (*2) [2..10])
80
ghci> sum $ filter (> 10) $ map (*2) [2..10]
80
```

定義はただの関数適用だが、優先順位が違う。空白による関数適用は優先順位が最も高く左結合で、`$` は最も低く右結合になる

`sqrt 3 + 4 + 9` は `(sqrt 3) + 4 + 9`

`sqrt $ 3 + 4 + 9` は `sqrt (3 + 4 + 9)`

`$` は、式の右端に閉じ括弧がある開き括弧と考えると読みやすい。括弧の入れ子を減らすときに使う

# 5.6 `$` を使った関数適用

- **関数適用そのものを関数として扱う**

```
ghci> map ($ 3) [(4+), (10*), (^2), sqrt]
[7.0,30.0,9.0,1.7320508075688772]
```

`($ 3)` は `$` のセクションで、関数を受け取ってその関数を 3 に適用する関数になる。これを `map` に渡すと、リスト中のすべての関数が 3 に適用される

| 関数 | `($ 3)` を適用 | 結果 |
|---|---|---|
| `(4+)` | `(4+) 3` | 7.0 |
| `(10*)` | `(10*) 3` | 30.0 |
| `(^2)` | `(^2) 3` | 9.0 |
| `sqrt` | `sqrt 3` | 1.732... |

# 目次


5.1 カリー化関数

5.2 高階実演

5.3 関数プログラマの道具箱

5.4 ラムダ式

5.5 畳み込み、見込みアリ！

5.6 `$` を使った関数適用

**5.7 関数合成**

# 5.7　関数合成

## 関数合成演算子 `.`

数学の $(f \circ g)(x) = f(g(x))$ にあたる演算子

```
(.) :: (b -> c) -> (a -> b) -> a -> c
f . g = \x -> f (g x)
```

`negate . (*3)` は、数を受け取って3倍し、符号を反転する関数になる

合成するには、`g` の返り値の型 `b` と `f` の引数の型が一致している必要がある。できた関数は `g` と同じ型を受け取り、`f` と同じ型を返す

```
ghci> map (\x -> negate (abs x)) [5,-3,-6,7,-3,2,-19,24]
[-5,-3,-6,-7,-3,-2,-19,-24]
ghci> map (negate . abs) [5,-3,-6,7,-3,2,-19,24]
[-5,-3,-6,-7,-3,-2,-19,-24]
```

# 5.7　関数合成

- **たくさんの関数を合成する**

関数合成は右結合なので、`f (g (z x))` は `(f . g . z) x` と書ける

```
ghci> map (\xs -> negate (sum (tail xs))) [[1..5],[3..6],[1..7]]
[-14,-15,-27]
ghci> map (negate . sum . tail) [[1..5],[3..6],[1..7]]
[-14,-15,-27]
```

`negate . sum . tail` は、`tail`、`sum`、`negate` の順に適用される。ラムダ式を書かなくても、その場で関数を組み立てて `map` に渡せる



# 5.7　関数合成

## ● 引数が複数ある関数を合成する

普通は、残り1引数になるまで部分適用しないと合成できない

```
sum (replicate 5 (max 6.7 8.9))
(sum . replicate 5) (max 6.7 8.9)
sum . replicate 5 $ max 6.7 8.9
```

3つとも同じ値 `44.5` になる。`replicate` は本来2引数だが、`replicate 5` と部分適用すると、1つの引数を5回繰り返したリストを作る1引数関数になる。`max 6.7 8.9` の結果が `replicate 5` に渡り、そのリストが `sum` に渡る

# 5.7　関数合成

- **括弧の多い式を関数合成で書き直す**

```
replicate 2 (product (map (*3) (zipWith max [1,2] [4,5])))
```

| 手順 | 書いた式 |
|---|---|
| ① 一番内側の関数と引数を書き出す | `zipWith max [1,2] [4,5]` |
| ② その前の関数を `$` でつなぐ | `map (*3) $ zipWith max [1,2] [4,5]` |
| ③ 次の関数を `.` で合成 | `product . map (*3) $ zipWith ...` |
| ④ さらに合成 | `replicate 2 . product . map (*3) $ zipWith ...` |

```
replicate 2 . product . map (*3) $ zipWith max [1,2] [4,5]
```

どちらの式も `[180,180]` になる。式の終わりに閉じ括弧が3つ以上並んでいたら、この手順で書き直せないか試してみる

# 5.7　関数合成

## ● ポイントフリースタイル

```
sum' :: (Num a) => [a] -> a
sum' xs = foldl (+) 0 xs
```

両辺の右端の `xs` を省略すると

```
sum' = foldl (+) 0
```

```
fn x = ceiling (negate (tan (cos (max 50 x))))
```

関数合成を使うとポイントフリーにできる

```
fn = ceiling . negate . tan . cos . max 50
```

`fn x = ...` の `x` のように関数定義に現れる一時変数をポイントと呼び、ポイントを書かずに関数を定義する書き方をポイントフリースタイルと呼ぶ

`fn` の右端の `x` は括弧の中にあるので、そのままでは消せない。関数合成で書き直すと消せる

ポイントフリーで書くと、どの関数をどの順に合成しているかを式から直接読み取れる

# 5.7　関数合成

- `oddSquareSum` を関数合成で書き換える

```
oddSquareSum :: Integer
oddSquareSum = sum (takeWhile (<10000) (filter odd (map (^2) [1..])))
```

関数合成を使うと、次のように書ける

```
oddSquareSum :: Integer
oddSquareSum = sum . takeWhile (<10000) . filter odd $ map (^2) [1..]
```

右から読むと、`map (^2) [1..]` に `filter odd`、`takeWhile (<10000)`、`sum` の順に適用される

合成のチェーンが長すぎると読みにくくなる。`let` で途中の結果に名前を付けて分割するのもよい

# 5.7　関数合成

## `$` と `.` を使い分ける

`.` は関数と関数をつなぎ、`$` は関数を値に適用する

```
oddSquareSum = sum . takeWhile (<10000) . filter odd $ map (^2) [1..]
oddSquareSum = sum . takeWhile (<10000) . filter odd . map (^2) $ [1..]
oddSquareSum = sum $ takeWhile (<10000) $ filter odd $ map (^2) [1..]
oddSquareSum = sum . takeWhile (<10000) . filter odd . map (^2) [1..]
```

上の3行は正しく動く。最後の行では `map (^2) [1..]` が先に結合してリスト（関数ではない）になり、`.` の右側にリストが来るので型エラーになる

# 5.7　Julia で書く関数合成

Julia は、関数合成の演算子 `∘` と、値を左から右へ流すパイプ `|>` を標準で持っている

```
map (negate . sum . tail) [[1..5],[3..6],[1..7]]
```

```
f = (-) ∘ sum ∘ (xs -> xs[2:end])
map(f, [1:5, 3:6, 1:7])
1:5 |> (xs -> xs[2:end]) |> sum |> (-)
```

2行目は `[-14, -15, -27]`、3行目は `-14` になる。`xs -> xs[2:end]` が `tail`、`(-)` が `negate` にあたる

`∘` は Haskell の `.` と同じく右の関数から適用し、`|>` は逆に左から順に適用する。Go には合成演算子がないので、関数呼び出しを入れ子にして書く

HANDS-ON

# H5　関数合成で書き換える（課題 問3）

4分

1. `report5.hs` の `oddSquareSum` を `$` と `.` を使って書く
2. 余裕があれば、発展として `fn` をポイントフリースタイルで書く
3. GHCi で型と値を確認する。`map (*3) . filter odd` がどんな型になるか予想してから確かめる

```
ghci> :r
ghci> oddSquareSum
166650
ghci> :t negate . abs
ghci> :t map (*3) . filter odd
ghci> map fn [0, 49, 50, 51, 100]
```

# 第5回 課題

`report5.hs` を完成させて提出する。型宣言は変更しない

| 問 | 内容 |
|---|---|
| 問1 | `zipWith'` を実装せよ（標準関数 `zipWith` は使用不可） |
| 問2 | `foldr` を使って `map'` と `filter'` を実装せよ |
| 問3 | `oddSquareSum` を `$` と `.` で書き換えよ（発展として `fn` をポイントフリーにする） |

提出前に `runghc report5.hs` を実行し、6件すべてが `[OK]` になることを確かめる。`[NG]` が出た問は、表示される `got` と `expected` を見比べて直す